

# **PROGRAMMING**

## **Topic 1: Programming Fundamentals**

### **1. What is a programming language?**

**Answer:** A programming language is a formal language used to write instructions that a computer can execute.

**Explanation:** It allows developers to communicate with computers and build software.

**Example:** C, C++, Java, and Python are programming languages.

### **2. What is a program?**

**Answer:** A program is a set of instructions written to perform a specific task.

**Explanation:** The computer follows these instructions to produce the required output.

**Example:** A calculator program performs arithmetic operations.

### **3. What is an algorithm?**

**Answer:** An algorithm is a step-by-step procedure for solving a problem.

**Explanation:** It defines the logical sequence of operations before implementation in code.

**Example:** Steps to find the largest number among three numbers.

### **4. What is a variable?**

**Answer:** A variable is a named storage location used to hold a value.

**Explanation:** Its value can change while a program is running.

**Example:** `int age = 18;`

### **5. What is a data type?**

**Answer:** A data type defines the kind of value a variable can store.

**Explanation:** It tells the programming language how the stored data should be interpreted.

**Example:** `int`, `float`, `char`, and `boolean`.

### **6. What is a constant?**

**Answer:** A constant is a value that should not change during program execution.

**Explanation:** Constants are useful for fixed values such as mathematical or configuration values.

**Example:** `const double PI = 3.14159;`

## 7. What is an operator in programming?

**Answer:** An operator is a symbol used to perform an operation on values or variables.

**Explanation:** Operators can perform arithmetic, comparison, logical, and other operations.

**Example:** `+, -, *, /, ==.`

## 8. What is a conditional statement?

**Answer:** A conditional statement executes code based on whether a condition is true or false.

**Explanation:** It allows a program to make decisions.

**Example:** `if (age >= 18)`

## 9. What is a loop?

**Answer:** A loop repeatedly executes a block of code while a condition is satisfied.

**Explanation:** Loops reduce the need to write the same code repeatedly.

**Example:** `for`, `while`, and `do-while`.

## 10. What is a function?

**Answer:** A function is a reusable block of code designed to perform a specific task.

**Explanation:** Functions improve code organization and reduce duplication.

**Example:** `calculateSum()` can return the sum of two numbers.

## 11. What is debugging?

**Answer:** Debugging is the process of finding and fixing errors in a program.

**Explanation:** Developers use debugging to identify why a program produces incorrect results or fails.

**Example:** Finding why a calculation returns the wrong value.

## **12. What is a syntax error?**

**Answer:** A syntax error occurs when code violates the rules of a programming language.

**Explanation:** The compiler or interpreter usually reports the error before the program can run correctly.

**Example:** Missing ; in languages such as C++.

## **13. What is a logical error?**

**Answer:** A logical error occurs when a program runs but produces an incorrect result.

**Explanation:** The syntax may be valid, but the implemented logic is wrong.

**Example:** Using + instead of \* in a calculation.

## **14. What is source code?**

**Answer:** Source code is the human-readable code written by a programmer.

**Explanation:** It contains instructions that are later compiled or interpreted for execution.

**Example:** A Python file containing `print("Hello")`.

## **15. What is the difference between a compiler and an interpreter?**

**Answer:** A compiler translates code before execution, while an interpreter executes code through an interpreter process.

**Explanation:** Languages can use different execution models, and some modern runtimes combine compilation and interpretation techniques.

**Example:** C++ commonly uses compilation; Python commonly uses an interpreter/runtime.

## **Topic 2: C Programming**

### **16. What is the C programming language?**

**Answer:** C is a general-purpose, procedural programming language.

**Explanation:** C is widely used for system software, embedded systems, operating systems, and performance-critical applications.

**Example:** Operating-system components and embedded firmware can be written in C.

### **17. What is the basic structure of a C program?**

**Answer:** A C program typically contains header files, functions, and the main() function.

**Explanation:** Execution normally begins from the main() function.

**Example:** `int main() { return 0; }`

### **18. What is the purpose of main() in C?**

**Answer:** main() is the entry point of a C program.

**Explanation:** Program execution begins from the main() function.

**Example:** `int main() { printf("Hello"); }`

### **19. What are header files in C?**

**Answer:** Header files contain declarations and definitions that can be reused by a C program.

**Explanation:** They provide access to functions and other features supplied by libraries.

**Example:** `#include <stdio.h>` provides standard input/output functions.

### **20. What is a variable in C?**

**Answer:** A variable is a named memory location used to store a value.

**Explanation:** The value stored in a variable can change during program execution.

**Example:** `int marks = 85;`

### **21. What are the basic data types in C?**

**Answer:** Common basic data types include int, char, float, and double.

**Explanation:** Data types determine what kind of data a variable can represent.

**Example:** `int age = 18;`

### **22. What is the difference between int and float in C?**

**Answer:** int stores whole numbers, while float stores floating-point numbers.

**Explanation:** Use int for values without decimal parts and float when decimal values are needed.

**Example:** `int age = 18; and float price = 99.5f;`

### **23. What is an array in C?**

**Answer:** An array stores multiple values of the same data type in contiguous memory locations.

**Explanation:** It allows related values to be accessed using an index.

**Example:** `int marks[5];`

### **24. What is a pointer in C?**

**Answer:** A pointer is a variable that stores the memory address of another variable.

**Explanation:** Pointers are important for dynamic memory, arrays, functions, and low-level programming.

**Example:** `int *p = &age;`

### **25. What do & and \* mean when working with pointers?**

**Answer:** & obtains an address, while \* accesses the value stored at an address.

**Explanation:** These operators are fundamental to pointer operations in C.

**Example:** `p = &age;` and `value = *p;`

### **26. What is a function in C?**

**Answer:** A function is a reusable block of code designed to perform a specific task.

**Explanation:** Functions make programs modular and reduce code repetition.

**Example:** `int add(int a, int b)`

### **27. What is recursion in C?**

**Answer:** Recursion occurs when a function calls itself to solve a problem.

**Explanation:** A recursive function needs a base condition to stop further calls.

**Example:** Calculating factorial using a recursive function.

### **28. What is dynamic memory allocation in C?**

**Answer:** Dynamic memory allocation allows memory to be allocated during program execution.

**Explanation:** Functions such as `malloc()`, `calloc()`, `realloc()`, and `free()` are used for managing dynamic memory.

**Example:** `int *p = malloc(5 * sizeof(int));`

### **29. What is a structure in C?**

**Answer:** A structure is a user-defined data type that groups related variables of different data types.

**Explanation:** Structures are useful for representing records or complex data.

**Example:** A Student structure can contain name, age, and marks.

### **30. What is the difference between a structure and an array in C?**

**Answer:** An array stores elements of the same type, while a structure can store members of different types.

**Explanation:** Arrays are useful for collections of similar data; structures represent related but different data.

**Example:** `int marks[5]` is an array; `struct Student` can contain name, age, and marks.

## **Topic 4: Java Programming**

### **46. What is Java?**

**Answer:** Java is a high-level, object-oriented programming language designed to be portable across platforms.

**Explanation:** Java programs run through the Java Virtual Machine (JVM), which supports the principle “write once, run anywhere.”

**Example:** Java is widely used for enterprise applications, backend systems, and Android-related development.

### **47. What are the main features of Java?**

**Answer:** Key features include object orientation, platform independence, automatic memory management, security, and multithreading.

**Explanation:** These features make Java suitable for large-scale and portable applications.

**Example:** The same compiled Java bytecode can run on different operating systems with a compatible JVM.

### **48. What is the JVM?**

**Answer:** JVM stands for Java Virtual Machine.

**Explanation:** The JVM executes Java bytecode and provides the runtime environment needed by Java applications.

**Example:** A .class file containing bytecode is executed by the JVM.

#### **49. What is the difference between JDK, JRE, and JVM?**

**Answer:** JDK is for developing Java applications, JRE provides the runtime environment, and JVM executes bytecode.

**Explanation:** JDK includes tools needed to develop and run Java programs.

**Example:** A developer uses the JDK to compile Java source code.

#### **50. What is a class in Java?**

**Answer:** A class is a blueprint that defines the data and behavior of objects.

**Explanation:** It contains fields, methods, constructors, and other members.

**Example:** `class Student { String name; }`

#### **51. What is an object in Java?**

**Answer:** An object is an instance of a class.

**Explanation:** Objects represent entities and contain their own state while using the behavior defined by the class.

**Example:** `Student s1 = new Student();`

#### **52. What is inheritance in Java?**

**Answer:** Inheritance allows one class to acquire accessible properties and methods from another class.

**Explanation:** It promotes code reuse and represents an “is-a” relationship.

**Example:** `class Dog extends Animal`

#### **53. What is method overloading in Java?**

**Answer:** Method overloading means defining multiple methods with the same name but different parameter lists.

**Explanation:** It provides multiple ways to perform related operations.

**Example:** `add(int, int)` and `add(double, double)`.

#### **54. What is method overriding in Java?**

**Answer:** Method overriding occurs when a subclass provides its own implementation of an inherited method.

**Explanation:** It is an important mechanism for run-time polymorphism.

**Example:** A Dog class overrides the sound() method inherited from Animal.

### **55. What is an interface in Java?**

**Answer:** An interface defines a contract that implementing classes agree to follow.

**Explanation:** Interfaces are commonly used to achieve abstraction and support multiple type inheritance.

**Example:** class Payment implements Payable

### **56. What is exception handling in Java?**

**Answer:** Exception handling manages abnormal conditions that occur during program execution.

**Explanation:** Java provides try, catch, finally, throw, and throws for handling exceptions.

**Example:** Handling division by zero with try-catch.

### **57. What is garbage collection in Java?**

**Answer:** Garbage collection automatically identifies and reclaims memory occupied by objects that are no longer reachable.

**Explanation:** It reduces the need for manual memory deallocation.

**Example:** An unreachable object may later have its memory reclaimed by the JVM.

### **58. What is multithreading in Java?**

**Answer:** Multithreading allows multiple threads to execute tasks concurrently within a program.

**Explanation:** It can improve responsiveness and resource utilization when tasks can run concurrently.

**Example:** A server can use separate threads to handle multiple client requests.

### **59. What is the Java Collections Framework?**

**Answer:** It is a set of interfaces and classes used to store and manipulate groups of objects.

**Explanation:** It provides reusable data structures and algorithms.

**Example:** ArrayList, HashSet, and HashMap.

## **60. Why is Java considered platform-independent?**

**Answer:** Java source code is compiled into bytecode that can run on any compatible JVM.

**Explanation:** The JVM abstracts many operating-system-specific details from the Java application.

**Example:** The same .class file can run on Windows or Linux with a suitable JVM.

## **Topic 5: Python Programming**

### **61. What is Python?**

**Answer:** Python is a high-level, general-purpose programming language known for its readable syntax.

**Explanation:** It is widely used in web development, automation, data science, AI, machine learning, and scripting.

**Example:** `print("Hello, World!")`

### **62. What are the main features of Python?**

**Answer:** Python offers readable syntax, dynamic typing, automatic memory management, extensive libraries, and support for multiple programming paradigms.

**Explanation:** These features make Python useful for both beginners and professional developers.

**Example:** Python is commonly used for automation and AI applications.

### **63. What is a variable in Python?**

**Answer:** A variable is a name that refers to a value or object.

**Explanation:** Python determines the type at runtime, so explicit type declaration is usually not required.

**Example:** `age = 18`

### **64. What are common built-in data types in Python?**

**Answer:** Common types include int, float, str, bool, list, tuple, set, and dict.

**Explanation:** Different data types represent different kinds of values and collections.

**Example:** name = "Alex" and marks = [80, 90, 85]

### **65. What is a list in Python?**

**Answer:** A list is an ordered, mutable collection of elements.

**Explanation:** Lists can contain multiple values and their contents can be changed after creation.

**Example:** skills = ["Python", "Java", "C++"]

### **66. What is a tuple in Python?**

**Answer:** A tuple is an ordered, immutable collection of elements.

**Explanation:** Once created, its elements cannot normally be changed.

**Example:** coordinates = (10, 20)

### **67. What is a dictionary in Python?**

**Answer:** A dictionary stores data as key-value pairs.

**Explanation:** It is useful when values need to be associated with unique keys.

**Example:** student = {"name": "Alex", "age": 18}

### **68. What is a set in Python?**

**Answer:** A set is an unordered collection of unique elements.

**Explanation:** Sets are useful for removing duplicates and performing mathematical set operations.

**Example:** skills = {"Python", "Java", "Python"} results in unique values.

### **69. What is a function in Python?**

**Answer:** A function is a reusable block of code that performs a specific task.

**Explanation:** Functions improve code organization and reduce repetition.

**Example:** def calculate\_total(price, tax):

### **70. What is a lambda function in Python?**

**Answer:** A lambda is a small anonymous function written using the lambda keyword.

**Explanation:** It is useful for short operations, especially when used with functions such as `map()` and `filter()`.

**Example:** `square = lambda x: x * x`

### 71. What is list comprehension in Python?

**Answer:** List comprehension is a concise way to create a list from an iterable.

**Explanation:** It can combine iteration and optional filtering in a compact expression.

**Example:** `[x * 2 for x in range(5)]`

### 72. What is exception handling in Python?

**Answer:** Exception handling manages runtime errors using constructs such as `try` and `except`.

**Explanation:** It allows a program to respond to errors without unexpectedly terminating.

**Example:** Handling invalid input using `try-except`.

### 73. What is a module in Python?

**Answer:** A module is a Python file containing reusable code such as functions, classes, or variables.

**Explanation:** Modules help organize large programs into manageable components.

**Example:** `import math`

### 74. What is a package in Python?

**Answer:** A package is a structured collection of Python modules organized under a common namespace.

**Explanation:** Packages help developers organize and distribute related functionality.

**Example:** `numpy` provides modules and functionality for numerical computing.

### 75. What is a virtual environment in Python?

**Answer:** A virtual environment is an isolated environment for a Python project and its dependencies.

**Explanation:** It prevents different projects from interfering with each other's package versions.

**Example:** `python -m venv myenv`

## **Topic 6: OOP Concepts**

### **76. What is Object-Oriented Programming (OOP)?**

**Answer:** OOP is a programming approach that organizes software using **objects and classes**.

**Explanation:** Objects combine data and the functions that operate on that data.

**Example:** A Student class can contain name, age, and a `displayDetails()` method.

### **77. What is encapsulation?**

**Answer:** Encapsulation means combining data and related methods inside a class and controlling access to the data.

**Explanation:** It helps protect data from unwanted direct access.

**Example:** A BankAccount class can keep balance private and provide `deposit()` and `withdraw()` methods.

### **78. What is abstraction?**

**Answer:** Abstraction means hiding unnecessary implementation details and showing only the essential features.

**Explanation:** The user uses the functionality without needing to know how it works internally.

**Example:** When using an ATM, you select “Withdraw” without knowing the internal banking process.

### **79. What is inheritance?**

**Answer:** Inheritance allows one class to acquire properties and methods from another class.

**Explanation:** It promotes code reuse and represents an **is-a relationship**.

**Example:** A Car class can inherit common features from a Vehicle class.

### **80. What is polymorphism?**

**Answer:** Polymorphism means “many forms,” where the same interface or method name can behave differently depending on the object.

**Explanation:** It allows flexible and reusable program design.

**Example:** A draw() method can behave differently for Circle, Rectangle, and Triangle.

### 81. What is method overriding?

**Answer:** Method overriding occurs when a child class provides its own implementation of a method inherited from its parent class.

**Explanation:** It is commonly used to achieve runtime polymorphism.

**Example:** A Dog class can override the sound() method of an Animal class.

### 82. What is method overloading?

**Answer:** Method overloading means using the same method name with different parameter lists.

**Explanation:** It allows a method to perform similar operations with different inputs.

**Example:** calculate(int a, int b) and calculate(int a, int b, int c).

### 83. What is the difference between composition and inheritance?

**Answer:** Inheritance represents an **is-a** relationship, while composition represents a **has-a** relationship.

**Explanation:** Composition builds a class using objects of other classes and often provides more flexible design.

**Example:** A Car **has an** Engine, so Car can use composition.

### 84. What is an interface?

**Answer:** An interface defines a contract that implementing classes must follow.

**Explanation:** It is useful for achieving abstraction and allowing different classes to follow the same set of behaviors.

**Example:** A Payment interface can define pay(), which can be implemented by UPIPayment, CardPayment, and WalletPayment.

### 85. What are the SOLID principles?

**Answer:** SOLID is a group of five principles used to design maintainable and flexible object-oriented software.

**Explanation:** SOLID stands for:

- **S** – Single Responsibility Principle
- **O** – Open/Closed Principle
- **L** – Liskov Substitution Principle
- **I** – Interface Segregation Principle
- **D** – Dependency Inversion Principle

**Example:** A class should ideally have one clear responsibility instead of handling unrelated tasks.

## **Topic 7: Data Structures & Algorithms**

### **86. What is a data structure?**

**Answer:** A data structure is a way of organizing and storing data so that it can be accessed and processed efficiently.

**Explanation:** Different data structures are suitable for different types of problems.

**Example:** An array can store a list of student marks.

### **87. What is an array?**

**Answer:** An array is a collection of elements stored in an ordered sequence, usually accessed using an index.

**Explanation:** Arrays are useful when we need to store multiple values of the same general type in a structured way.

**Example:** marks = [80, 75, 90, 85]

### **88. What is a linked list?**

**Answer:** A linked list is a data structure made up of nodes, where each node stores data and a link to another node.

**Explanation:** Unlike a traditional array, linked-list elements do not need to be stored next to each other in memory.

**Example:** 10 → 20 → 30 → 40

### **89. What is a stack?**

**Answer:** A stack is a linear data structure that follows **LIFO (Last In, First Out)**.

**Explanation:** The most recently added element is removed first.

**Example:** The browser's **Back** operation can be modeled using a stack.

### **90. What is a queue?**

**Answer:** A queue is a linear data structure that generally follows **FIFO (First In, First Out)**.

**Explanation:** The element added first is normally processed first.

**Example:** A printer queue processes print jobs in order.

### 91. What is a hash table?

**Answer:** A hash table stores data using **key-value pairs** and uses a hash function to determine where values are stored.

**Explanation:** It can provide very fast average-case lookup, insertion, and deletion.

**Example:** `{"id": 101, "name": "Arun"}`

### 92. What is a tree data structure?

**Answer:** A tree is a hierarchical data structure consisting of nodes connected through relationships such as parent and child.

**Explanation:** Trees are widely used for representing hierarchical information.

**Example:** A file system can be represented as a tree of folders and files.

### 93. What is a graph?

**Answer:** A graph is a data structure consisting of **vertices (nodes)** and **edges (connections)**.

**Explanation:** Graphs are useful for representing relationships and networks.

**Example:** A social network can represent users as nodes and friendships as edges.

### 94. What is an algorithm?

**Answer:** An algorithm is a finite sequence of well-defined steps used to solve a problem.

**Explanation:** A good algorithm should solve the required problem correctly and efficiently.

**Example:** Binary search is an algorithm for finding an element in a sorted collection.

### 95. What is time complexity?

**Answer:** Time complexity describes how an algorithm's running time grows as the input size increases.

**Explanation: Big-O notation** is commonly used to express this growth.

**Example:** Linear search has  **$O(n)$**  time complexity in the worst case, while binary search has  **$O(\log n)$**  on a sorted collection.

## **Topic 8: Modern Programming / APIs / Git**

### **96. What is an API?**

**Answer:** API stands for **Application Programming Interface**. It allows different software applications to communicate with each other.

**Explanation:** An application can request data or services from another application through an API.

**Example:** A weather application can use a weather API to get current weather data.

### **97. What is a REST API?**

**Answer:** A REST API is a web API that follows REST principles and commonly uses HTTP methods such as GET, POST, PUT, and DELETE.

**Explanation:** REST APIs are widely used for communication between web applications, mobile apps, and backend servers.

**Example:** GET /users can request a list of users.

### **98. What is JSON?**

**Answer:** JSON stands for **JavaScript Object Notation**. It is a lightweight text format commonly used for exchanging structured data.

**Explanation:** JSON is easy for both humans and programs to read and process.

**Example:**

```
{  
  "name": "Arun",  
  "skill": "Python"  
}
```

### **99. What is Git?**

**Answer:** Git is a **distributed version control system** used to track changes in source code.

**Explanation:** Developers can save versions of their code, compare changes, create branches, and collaborate with other developers.

**Example:** git commit saves a set of changes to the local Git repository.

### **100. What is GitHub?**

**Answer:** GitHub is a platform for hosting Git repositories and collaborating on software projects.

**Explanation:** Developers can store code online, work with branches, review changes, manage issues, and collaborate with teams.

**Example:** A student can upload a Python project to GitHub and share the repository with a team member.